

TUGAS PRAKTIKUM PERTEMUAN 11
ADJACENCY MATRIX & LIST

MICHAEL HENDRA

J0403251156



PROGRAM STUDI TEKNOLOGI REKAYASA PERANGKAT LUNAK
SEKOLAH VOKASI
INSTITUT PERTANIAN BOGOR
BOGOR
2026

1. Source Code

```
2. # NAMA : Michael Hendra
3. # NIM : J0403251156
4. # KELAS: TPL A1
5.
6. # PRAKTIKUM 1 - Membuat Adjacency Matrix
7. matrix = [
8.     [0, 1, 1, 0], # 0 terhubung dengan 1, 2
9.     [1, 0, 1, 0], # 1 terhubung dengan 0, 2
10.    [1, 1, 0, 1], # 2 terhubung dengan 0, 1
11.    [0, 0, 1, 0] # 3 terhubung dengan 2
12. ]
13.
14. def createMatrix(V, edges):
15.     # Membuat matrix V x V berisi 0
16.     matrix = [[0 for _ in range(V)] for _ in range(V)]
17.
18.     # Menambahkan edge
19.     for edge in edges:
20.         u = edge[0]
21.         v = edge[1]
22.         matrix[u][v] = 1
23.         matrix[v][u] = 1
24.     return matrix
25.
26. if __name__ == "__main__":
27.     V = 4
28.     # daftar edge
29.     edges = [[0, 1], [0, 2], [1, 2], [2, 3]]
30.
31.     # membuat adjacency matrix
32.     matrix = createMatrix(V, edges)
33.     print("PRAKTIKUM 1: Adjacency Matrix Representation:")
34.
35.     for row in matrix:
36.         print(row)
37.
38. #PRAKTIKUM 2 - Membuat Adjacency List
39. graph = {
40.     "A": ["B", "C"],
41.     "B": ["A", "D"],
42.     "C": ["A", "D"],
43.     "D": ["B", "C"]
44. }
45.
46. def createGraph(V, edges):
47.     adj = [[] for _ in range(V)]
48.     # Add each edge to adjacency list
```

```

49.     for it in edges:
50.         u = it[0]
51.         v = it[1]
52.         adj[u].append(v)
53.         adj[v].append(u) # karena undirected graph
54.     return adj
55.
56. if __name__ == "__main__":
57.     V = 4
58.     # List of edges (u, v)
59.     edges = [[0,1], [0,2], [1,3], [2,3]]
60.
61.     # Build the graph using edges
62.     adj = createGraph(V, edges)
63.     print("PRAKTIKUM 2: Adjacency List Representation:")
64.
65.     for i in range(V):
66.         # Print the vertex
67.         print(f'{i}: ', end=" ")
68.
69.         for j in adj[i]:
70.             # Print its adjacent
71.             print(j, end=" ")
72.         print()
73.
74. # PRAKTIKUM 3 - Konversi Matrix ke List
75. matrix = [ [0,1,1,0], [1,0,1,0], [1,1,0,1], [0,0,1,0] ]
76. list = {
77.     0 : [1, 2],
78.     1 : [0, 2],
79.     2 : [0, 1, 3],
80.     3 : [2]
81. }
82.
83. # PRAKTIKUM 4 – Studi Kasus Dunia Nyata (Digit Akhir NIM: 6)
84. # Langkah 1: Digit akhir = 6 (Studi kasus = Peta Kota)
85.
86. # Langkah 2: Tentukan Node dan Edge
87. # node = ["Bogor", "Jakarta Barat", "Jakarta Selatan", "Senen", "Tangerang"]
88. # Edge = Jalan penghubung
89.
90. # Langkah 3: Gambar Desain Graph
91. # Langkah 3 Praktikum 4 P11.drawio.png
92.
93. # Langkah 4: Implementasi dalam Python
94. graph = {
95.     "Jakarta Selatan": ["Jakarta Barat", "Jakarta Pusat", "Tangerang", "Bogor"],
96.     "Jakarta Barat": ["Jakarta Selatan", "Jakarta Pusat", "Tangerang"],

```

```

97.     "Jakarta Pusat": ["Jakarta Selatan", "Jakarta Barat"],
98.     "Tangerang": ["Jakarta Selatan", "Jakarta Barat"],
99.     "Bogor": ["Jakarta Selatan"]
100. }
101.
102.     matrix = [
103.         [0, 1, 1, 1, 1],
104.         [1, 0, 1, 1, 0],
105.         [1, 1, 0, 0, 0],
106.         [1, 1, 0, 0, 0],
107.         [1, 0, 0, 0, 0]
108.     ]
109.
110.     nodes = [
111.         "Jakarta Selatan",
112.         "Jakarta Barat",
113.         "Jakarta Pusat",
114.         "Tangerang",
115.         "Bogor"
116.     ]
117.     # LANGKAH 5 - Tampilkan Output Program
118.
119.     def createGraphs(edges):
120.         graph = {}
121.         # Add each edge to adjacency list
122.         for u, v in edges:
123.             # jika node belum ada
124.             if u not in graph:
125.                 graph[u] = []
126.
127.             if v not in graph:
128.                 graph[v] = []
129.             # karena undirected graph
130.             graph[u].append(v)
131.             graph[v].append(u)
132.         return graph
133.
134.     if __name__ == "__main__":
135.         # List of edges (u, v)
136.         edges = [
137.             ["Jakarta Selatan", "Jakarta Barat"],
138.             ["Jakarta Selatan", "Jakarta Pusat"],
139.             ["Jakarta Selatan", "Tangerang"],
140.             ["Jakarta Selatan", "Bogor"],
141.             ["Jakarta Barat", "Jakarta Pusat"],
142.             ["Jakarta Barat", "Tangerang"]
143.         ]
144.

```

```

145.         # Build the graph using edges
146.         adj = createGraphs(edges)
147.         print("PRAKTIKUM 4 Langkah 5.1: Adjacency List Representation:")
148.
149.         for node in adj:
150.             # Print the vertex
151.             print(f"{node}:", end=" ")
152.
153.             # Print its adjacent
154.             for neighbor in adj[node]:
155.                 print(neighbor, end=" ")
156.             print()
157.
158.         print("PRAKTIKUM 4 Langkah 5.2: Adjacency Matrix Representation:")
159.         for row in matrix:
160.             print(row)
161.
162.         print("PRAKTIKUM 4 Langkah 5.3: Nama Node")
163.         for node in nodes:
164.             print("-", node)
165.
166.         print("PRAKTIKUM 4 Langkah 5.4: Hubungan Antar Node")
167.         for node in graph:
168.             for neighbor in graph[node]:
169.                 print(node, "<->", neighbor)
170.

```

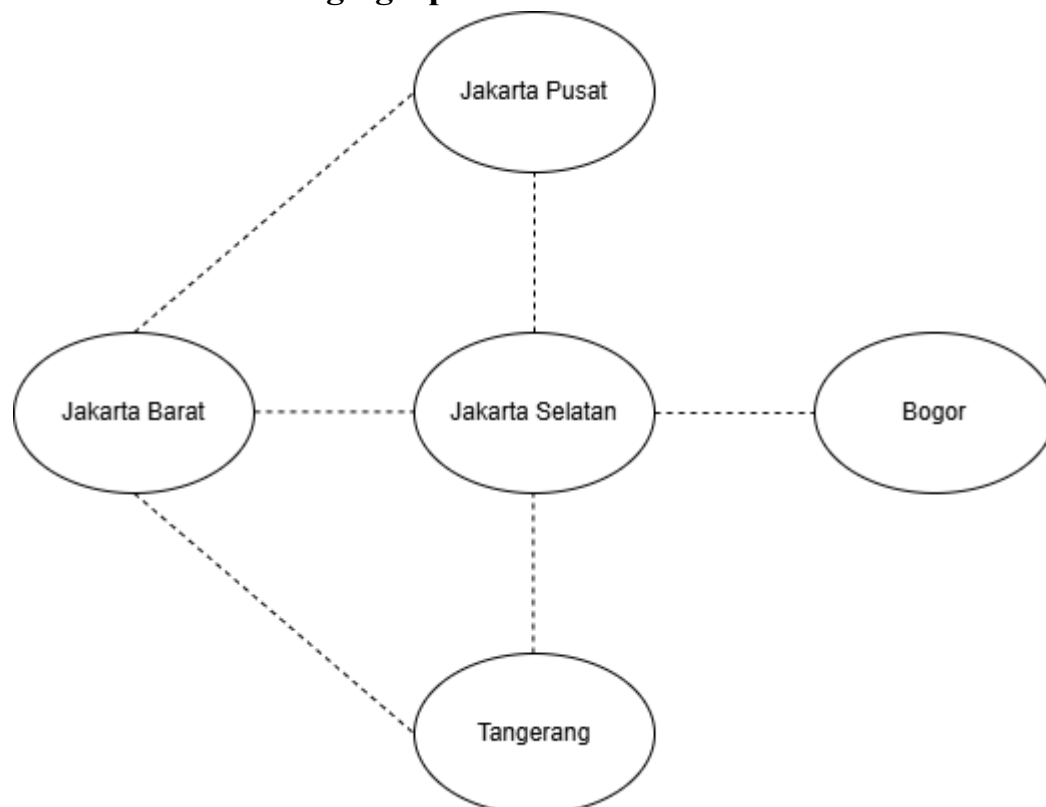
2. Output Program

```

PRAKTIKUM 1: Adjacency Matrix Representation:
[0, 1, 1, 0]
[1, 0, 1, 0]
[1, 1, 0, 1]
[0, 0, 1, 0]
PRAKTIKUM 2: Adjacency List Representation:
0: 1 2
1: 0 3
2: 0 3
3: 1 2
PRAKTIKUM 4 Langkah 5.1: Adjacency List Representation:
Jakarta Selatan: Jakarta Barat Jakarta Pusat Tangerang Bogor
Jakarta Barat: Jakarta Selatan Jakarta Pusat Tangerang
Jakarta Pusat: Jakarta Selatan Jakarta Barat
Tangerang: Jakarta Selatan Jakarta Barat
Bogor: Jakarta Selatan
PRAKTIKUM 4 Langkah 5.2: Adjacency Matrix Representation:
[0, 1, 1, 1, 1]
[1, 0, 1, 1, 0]
[1, 1, 0, 0, 0]
[1, 1, 0, 0, 0]
[1, 0, 0, 0, 0]
PRAKTIKUM 4 Langkah 5.3: Nama Node
- Jakarta Selatan
- Jakarta Barat
- Jakarta Pusat
- Tangerang
- Bogor
PRAKTIKUM 4 Langkah 5.4: Hubungan Antar Node
Jakarta Selatan <-> Jakarta Barat
Jakarta Selatan <-> Jakarta Pusat
Jakarta Selatan <-> Tangerang
Jakarta Selatan <-> Bogor
Jakarta Barat <-> Jakarta Selatan
Jakarta Barat <-> Jakarta Pusat
Jakarta Barat <-> Tangerang
Jakarta Pusat <-> Jakarta Selatan
Jakarta Pusat <-> Jakarta Barat
Tangerang <-> Jakarta Selatan
Tangerang <-> Jakarta Barat
Bogor <-> Jakarta Selatan

```

3. Gambar dan design graph



4. Penjelasan node dan edge

Node nya ada 6 yaitu Bogor, Jakarta Barat, Jakarta Pusat, Tangerang, Bogor

Edgenya garis penghubung antar kota:

- Jakarta Selatan ↔ Jakarta Barat
- Jakarta Selatan ↔ Jakarta Pusat
- Jakarta Selatan ↔ Tangerang
- Jakarta Selatan ↔ Bogor
- Jakarta Barat ↔ Jakarta Pusat
- Jakarta Barat ↔ Tangerang

5. Analisis adjacency list vs matrix

Adjacency list menyimpan daftar tetangga dari setiap node (cocok untuk graph besar dan mudah melihat node yang saling terhubung. Sedangkan Adjacency matrix menyimpan hubungan graph dalam bentuk tabel 2 dimensi (pengecekan hubungan antar node sangat cepat dan cocok untuk graph dengan banyak edge (dense graph)).

6. Kesimpulan singkat

Pada Praktikum 4, graph digunakan untuk merepresentasikan hubungan antar kota pada studi kasus peta kota. Setiap kota direpresentasikan sebagai node, sedangkan jalan penghubung direpresentasikan sebagai edge.

Graph yang dibuat termasuk undirected graph karena hubungan antar kota dapat dilalui dua arah. Representasi graph diimplementasikan menggunakan adjacency list dan adjacency matrix dalam Python.